Challenge 1:

To complete the challenge, the test must ensure that an object's state is reset after performing 3+ consecutive surges.

TC	Input	Expected Output
1	4 Spell Surges	Wizard.State == IDLE and manaStore == 600

Challenge 2:

To complete the challenge, the test must check that attempting to surge without channeling first will lead to a state exception.

2	SpellSurge()	InvalidStateException

Challenge 3:

To complete the challenge, the test has to ensure that investing mana amounts leading to negative throws an exception

CSV Values:
50,50,20,20
100,200,10,10

3	int surgeMana1, int surgeMana2, int spellMana1, int spellMana2	ManaInvestedException

Challenge 4:

Ensure that the Archmage receiving damage with an active barrier does not break the superclass rule of non-negative mana

4	Spellcaster wizard = new Archmage(10, 100);	ManaInvestedException

Challenge 5:

Ensure that the Archmage's spellSurge does not override the parent's (wizard) state.

5	spellSurge(100) on Archmage (yielding > 200 power)	Stance remains CHANNELING and does not incorrectly reset to IDLE

Challenge 6:

The test must ensure that state is consistent across operations on both base class wizard and its child class Archmage.

6	Identical instantiation and operations on Wizard and Archmage	manaStore, hp, level, and castSpell results match perfectly

Challenge 7:

The test must catch when integer parameters are swapped internally when sending mana.

7	transferMana(recipient, amount = 100, priorityTier = 1)	Sender manaStore decreases by 100; recipient increases by 100

Challenge 8:

The test must show how static type binding routes the object to different methods based on the compile-time reference type.

8	castSpell((Object) spell) vs	Different integer return values

	castSpell((Spell) spell)	(e.g., 10 vs 40)